

Lecture 4

1. 2 views of Linguistic structures

- **View 1 - CFG!!**

- relating actual sentences with CFGs in such a way for ex with **verb/noun/adj as variables**, connected through grammar rules, and the **actual words as terminals**

- **View 2 - Dependency structure**

- it shows all the words which modify/depend on other words
- for ex. look in the kitchen. "the" depends on "kitchen"
- **Prepositional phrases** can cause ambiguity if you don't correctly identify what it modifies
 - A prepositional phrase (PP) is a chunk of words that starts with a preposition (like in, on, at, under, with, to, from, for)

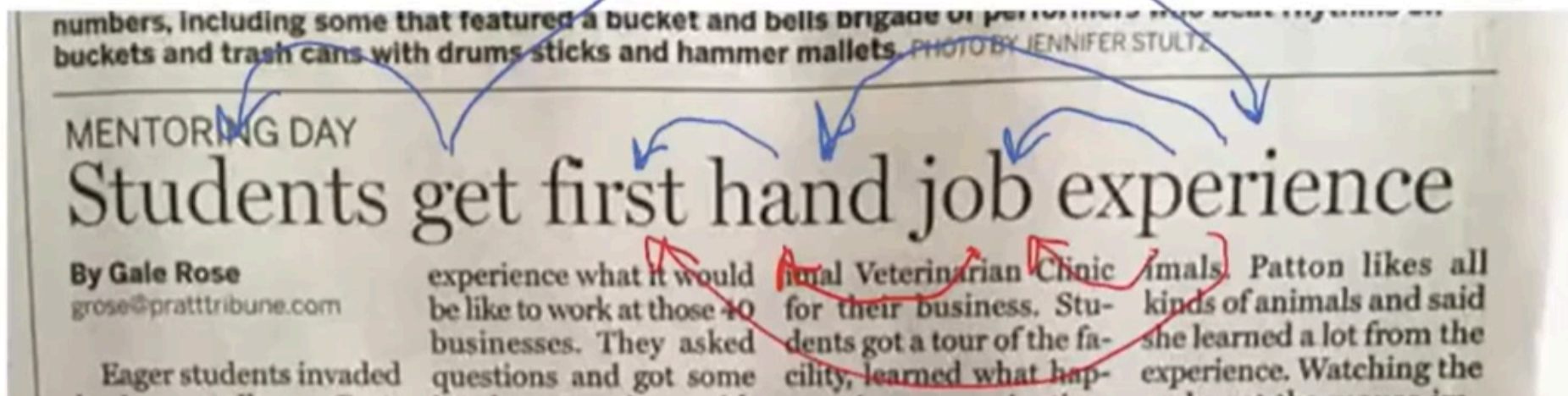
■
Scientists count whales from space



Scientists count whales from space



- need to identify what "from space (PP)" modifies
- with **K** number of Prepositional phrase, there are $Cat(n)$ number of parse structures, ways of explaining the dependency structure
- **Coordination scope ambiguity**: comes from how you interpret the sentence, like if you put a comma/and/or in the sentence, it changes the meaning.
- **Adjectival/Adverbial Modifier ambiguity**:



■ HAHAHHA

○ Verb Phrase attachment ambiguity:

Mutilated body washes up
on Rio beach to be used for
Olympics beach volleyball

○ **Convention:** we take the arrow as the head pointing to the dependent(the word depending on the head)

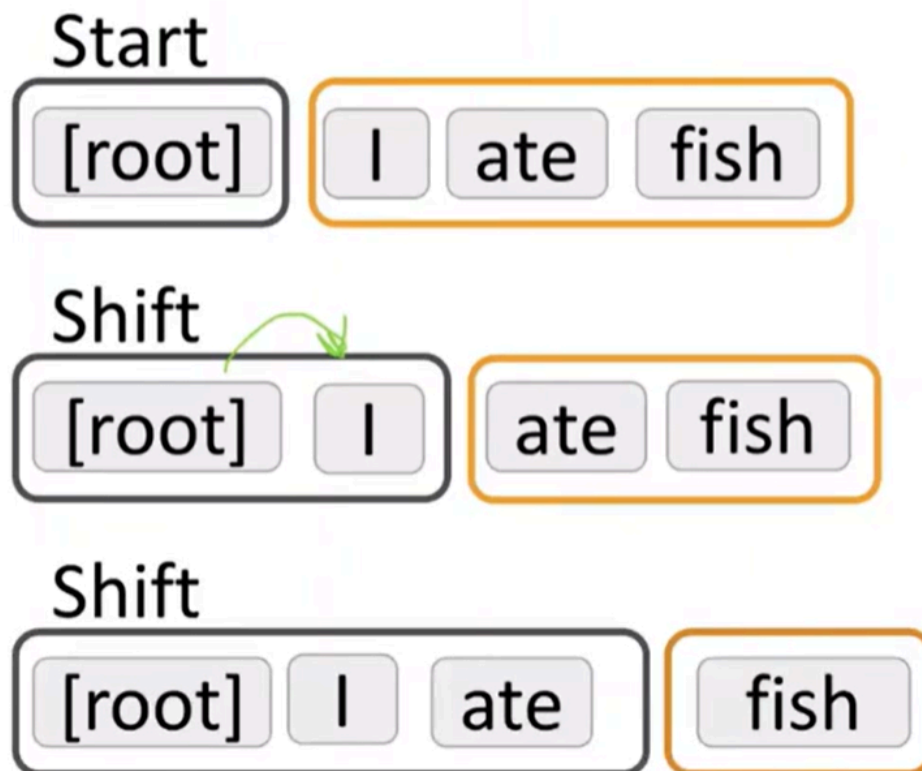
2. Dependency Parsing

- **Constructing**
 - a. we choose for each word what words it depends on
 - b. then we choose the **Root** of the dependency tree, where it doesn't depend on any other words
 - c. we don't want the occurrence of cycles.
- **Projectivity parsing:** means that when the words are in their original linear order, there are no crossing between arcs of the dependency arrows
 - And Dependency corresponding to a CFG Tree must be **Projective**
- **Methods:** of building dependency Parsers
 - through Dynamic Programming, through $O(n^3)$
 - through Graph Algorithms
 - through Minimum spanning tree (最小生成树)
 - **Transition-based parsing:**
 - it is a greedy method. takes the method of **bottom-up** actions (starting from small pieces and building up),
 - And consists of
 - A stack σ , top at right, with pushed Root initially
 - Buffer β , at the top left, starts with the input sentence
 - A set of **Dependency arcs A**, starts empty
 - And a set of actions includes like {Shift, Left-Arc, Right=Arc}
 - So the algo ends when $\sigma = ROOT$ and $\beta = \emptyset$
 - Example

Arc-standard transition-based parser

(there are other transition schemes ...)

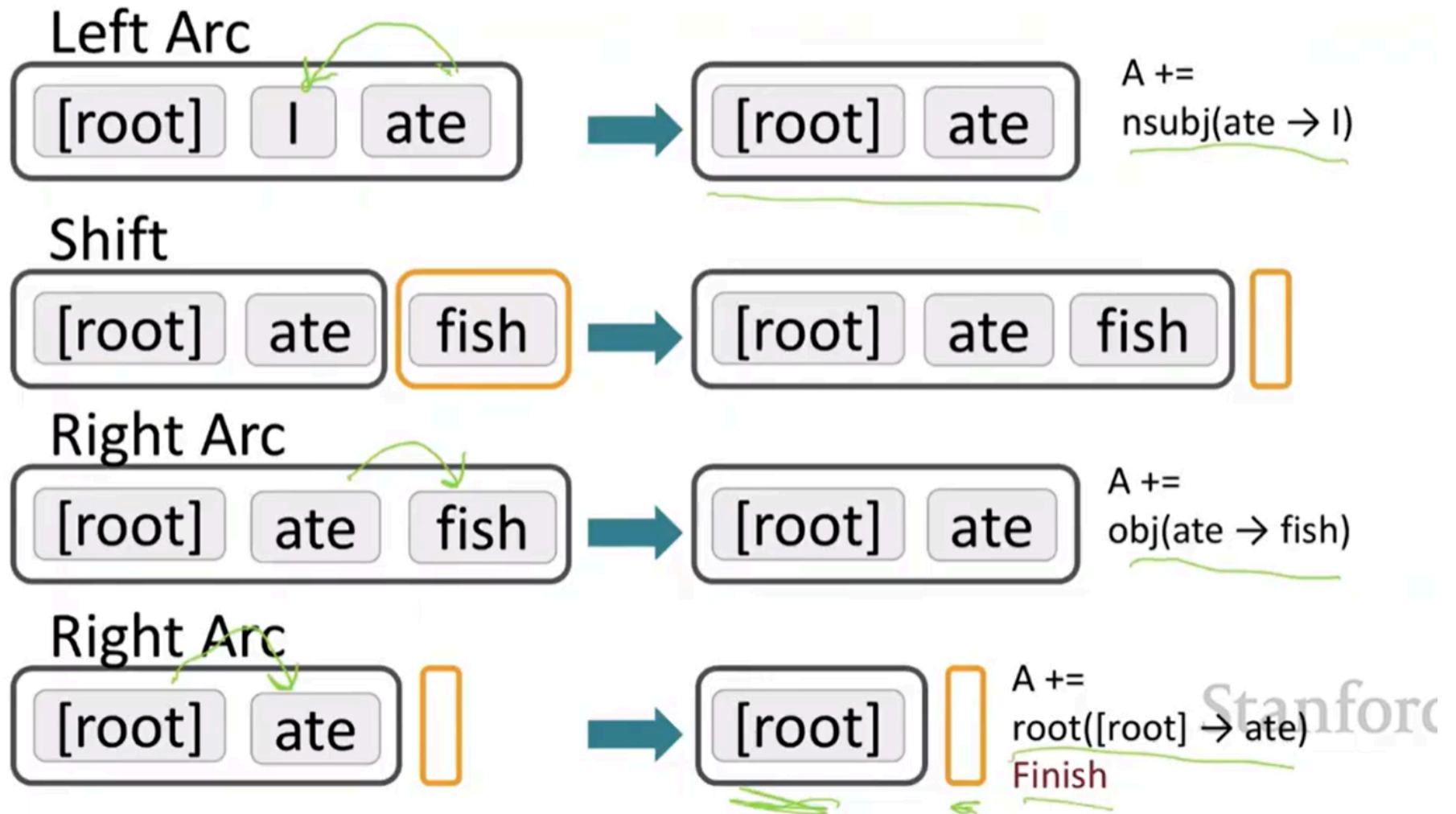
Analysis of “I ate fish”



Start: $\sigma = [\text{ROOT}]$, $\beta = w_1, \dots, w_n$, $A = \emptyset$

1. Shift $\sigma, w_i | \beta, A \rightarrow \sigma | w_i, \beta, A$
2. Left-Arc_r $\sigma | w_i | w_j, \beta, A \rightarrow \sigma | w_j, \beta, A \cup \{r(w_j, w_i)\}$
3. Right-Arc_r $\sigma | w_i | w_j, \beta, A \rightarrow \sigma | w_i, \beta, A \cup \{r(w_i, w_j)\}$

Finish: $\sigma = [w]$, $\beta = \emptyset$



- So how to predict the next action?
 - Answer is ML, use neural nets to train classifier to predict the next action